Sri Lanka Institute of Information Technology



Data Warehousing and Business Intelligence | IT3101
B.Sc. (Hons) in Information Technology
Olist E Commerce Data Warehouse and Business Intelligence

IT3101 Assignment Report

StudentID	Name
IT24103864	Amarakoon D.D
IT24104273	Nipun J.A.D
IT24102286	Hisham M.
IT24102567	Kodithuwakku I.S.H

Table of Contents

Section	Page
Introduction	3
1. Business Context and Dataset	**3**
2. Source Files and Preparation	**3**
2.1 Source Descriptions and Integration Keys	3
2.2 Data Preparation Steps	4
3. Architecture	**4**
3.1 Architecture Diagram	5
3.2 Explanation of Architecture Components	5
4. Dimensional Model	**6**
4.1 Star Schema, Snowflake Schema and Relationships	6
4.2 Date Hierarchy	6
4.3 Table Descriptions	6
4.4 Snowflake Schema Diagram	7
4.5 Star Schema Diagram	8
4.6 Design Assumptions	9
5. ETL and Checks	**10**
5.1 ETL Workflow and Loading Order	10
5.2 Transformation Logic	10
5.3 Validation Checks and Evidence	11
5.4 Loading Sequence	12
5.5 Validation Results	12
6. Sales Analysis Mart	**12**
6.1 Target Users	12
6.2 Implementation	12
6.3 Analytical Benefits	13
7. Power BI Report	**13**
7.1 Measures and Dashboard Overview	13
7.2 Executive Summary Dashboard	14
7.3 Trend Analysis Dashboard	14
7.4 Interactive Analysis Dashboard	15
8. Findings and Recommendations	**15**
Group Contribution and Submitted Files	16
Use of AI Tools	16
References	16

Introduction

Our group built an Olist sales warehouse in Microsoft SQL Server and explored it in Power BI. The operational data arrives as linked CSV files containing customers, orders, items, products, sellers and translated product categories. Six files support the sales fact, and a separate payments source was staged. The main outputs are an item-grain FactSales table, a normalized product-category branch, a sales analysis view and three dashboard pages. The report follows the eight assignment tasks in the order we completed them.

1 Business context and dataset

The dataset represents orders placed in an online marketplace. A customer places an order, an order may contain several products, and each item is associated with a seller. Payment and review records describe other parts of the same transaction. We chose it because these connected operational files can answer practical questions:

- **which categories sell most?**
- **where customers are located?**
- **How sales change over time?**
- **How much freight contributes to each order?**

The supplied archive contains nine CSV files. The figures in Table 1 count data records, excluding column headers. We used the supplied archive as the dataset source; the original publication URL should be added if available to the group.

File	Rows	Representative attributes
customers	99,441	customer_id, customer_unique_id, city, state
geolocation	1,000,163	zip prefix, latitude, longitude, city
order_items	112,650	order_id, order_item_id, product_id, seller_id, price, freight
order_payments	103,886	order_id, sequence, type, installments, value
order_reviews	99,224	review_id, order_id, score, comment
orders	99,441	order_id, customer_id, status, purchase and delivery dates
products	32,951	product_id, category, dimensions and weight
sellers	3,095	seller_id, location
category translation	71	Portuguese category, English category

An order may include more than one item. We therefore used one item within one order as the grain of the sales fact. Item price and freight can be summed at this grain. We count orders with DISTINCTCOUNT(order_id), since counting item rows would overstate the number of orders.

2 Source files and preparation

All nine input files are comma-separated CSVs. Six feed the sales mart, one is staged for separate payments analysis, and two were assessed but not integrated. Table 2 gives the role, available attributes, join path and preparation decision for each file. A relationship is listed even when a source was not joined into this version of the warehouse.

Source descriptions and integration keys

Source	Role	Selected attributes	Relationship and preparation
Orders	Operational orders and status	order_id, customer_id, purchase timestamp, status, delivery dates	order_id → items; customer_id → customers. Parse purchase dates; allow missing optional delivery dates.
Order items	One row per item sold	order_id, order_item_id, product_id, seller_id, price, freight_value	order_id → orders; product_id → products; seller_id → sellers. Convert prices and freight to decimal.

Customers	Customer IDs and location	customer_id, unique ID, city, state, ZIP prefix	customer_id → orders. Trim IDs/city; uppercase state; convert ZIP prefix.
Products	Item attributes and category	product_id, Portuguese category, weight, size	product_id → items; category → translation. Handle blank category; validate numeric types.
Sellers	Seller IDs and location	seller_id, city, state, ZIP prefix	seller_id → items. Trim IDs/city; uppercase state; convert ZIP prefix.
Translation	Portuguese to English labels	product_category_name, product_category_name_english	category name → products. Correct generic import headers; use English label where available.
Payments	Order payment records	order_id, payment_sequential, payment_type, payment_value	order_id → orders. Staged separately; do not join payment rows directly to item prices.
Reviews	Order-level feedback	review_id, order_id, review_score, comments	order_id → orders. Profiled as available; not staged in this sales mart.
Geolocation	Locations by postal prefix	ZIP prefix, latitude, longitude, city, state	ZIP prefix → customer/seller location. Not joined: many rows per prefix may multiply facts.

The integration begins with imported staging tables, not direct CSV-to-dashboard joins. Keys in the source files connect the operational entities; customer, product, seller and date keys then identify conformed warehouse dimensions. Category labels are normalized into DimCategory in the snowflake design. We preserve order_id and order_item_id in the sales fact to trace an item back to its source.

Connection	**Join key**	**Purpose**
orders → customers	customer_id	Customer and location per order
order_items → orders	order_id	Item to purchase and status
order_items → products	product_id	Product category analysis
order_items → sellers	seller_id	Seller analysis
products → translation	product_category_name	English category display
payments → orders	order_id	Staged separately; not joined into the sales mart

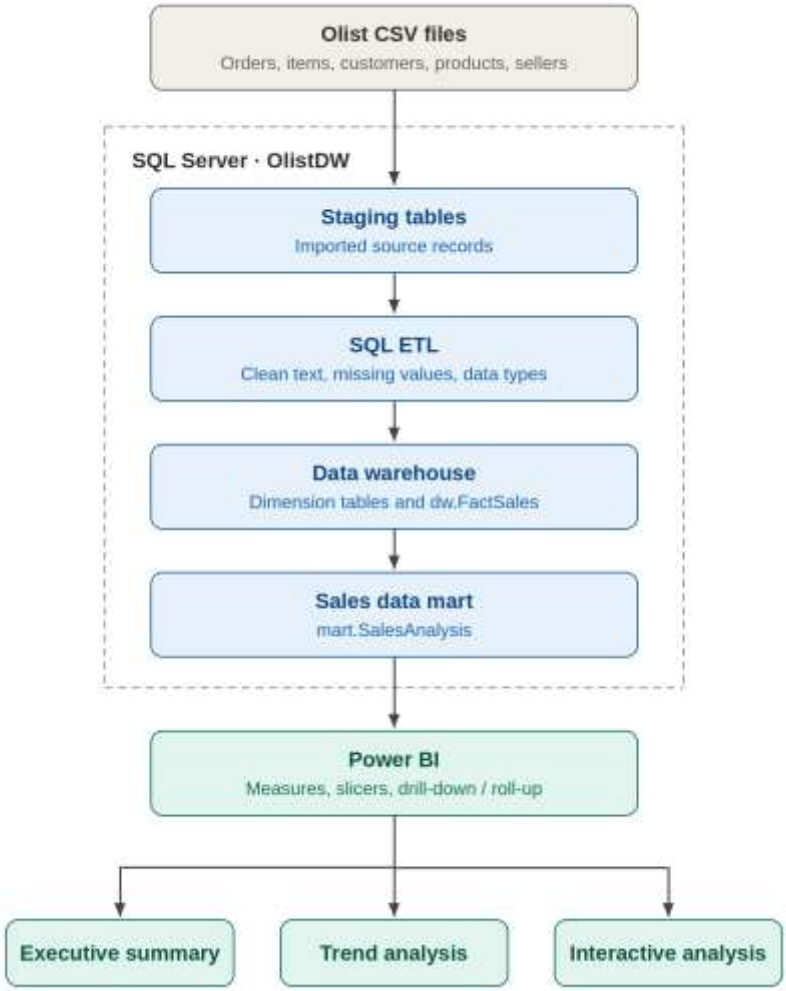
During the load, we removed surrounding spaces from text fields and normalized state codes to uppercase. Empty category values were assigned a readable fallback, while translated category names were used when present. TRY_CONVERT was used where source text had to become a date or number. Two import issues required attention: optional order dates could be blank, and product weight needed a numeric type large enough for the values in the file. The translation CSV was initially staged with generic column names, which we accounted for in the product load.

Data preparation steps

- Inspected headers, sample records and inferred column types.
- Allowed blanks in optional order timestamps.
- Corrected the category translation column mapping.
- Converted dates and numeric fields to suitable types.
- Trimmed surrounding spaces from identifiers and text.
- Standardized state codes to uppercase.
- Handled blank categories and missing translations.
- Checked row counts and key matches after import.

3 Architecture

Figure 1 shows our processing path. SSMS imports CSV data into staging tables, SQL transformations prepare the keys and descriptions, and dimension loads precede fact loads. The warehouse keeps the item-level sales data in dw.FactSales, while mart.SalesAnalysis exposes the attributes used for reporting. Power BI imports that mart and calculates delivered sales, distinct orders, freight and item counts. Payments is handled separately, so a many-to-many join cannot inflate item sales.



What each component does

- **Source layer**-Orders, items, customers, products, sellers and translations provide the six sales entities. Payments is a separate transactional source. Reviews and geolocation were examined but have no role in the current sales mart.
- **Integration layer**- SSMS reads CSV records into dbo staging tables. SQL transforms identifiers, dates, numeric values and category labels. The loading scripts match each source record to the warehouse keys.

- **Storage layer**- OlistDW retains staging data and the dw dimensions and facts. FactSales has an order-item grain. DimProduct points to DimCategory. FactPayments has its own row grain, keeping payment sums apart from sales.
- **Mart layer-**mart.SalesAnalysis provides the sales and marketing reporting fields in one view, including customer state, seller state, translated category, purchase date, item price and freight.
- **Presentation layer-**Power BI imports the mart, computes delivered measures and displays summary, trend and interactive report pages. Year, quarter and month provide drill-down and roll-up.

4 Dimensional model

We model an e-commerce sale at **one row per order item**. FactSales stores item price, freight and item count, plus the order status and foreign keys to customer, product, seller and purchase date. The original model was a star schema. We then created dw.DimCategory and linked it to DimProduct by category_key, producing the snowflake branch shown in Figure 2. Customer and seller state remain descriptive attributes of their respective dimensions.

Star schema,SnowFlake Schema and relationships

The fact table is linked to DimCustomer, DimProduct, DimSeller and DimDate through foreign keys. DimProduct also references DimCategory. The primary keys on all six tables and the implemented FactSales foreign keys were checked in SSMS result screens; the product-to-category relationship should be captured in the final snowflake diagram screenshot. The order_id and order_item_id pair identifies each source item.

Date hierarchy

DimDate provides purchase year, quarter, month number, month name and day for each full_date. This supports a Year → Quarter → Month → Day hierarchy. We used the same sequence in the Power BI drill visual. Aggregating item prices by month, quarter or year rolls the fact up without changing its underlying grain.

Warehouse object	Business meaning and selected fields
dw.FactSales	item grain; order_id, order_item_id, keys, order_status, price, freight_value, item_count
dw.DimCustomer	customer_key, customer_id, customer_unique_id, city, state, ZIP prefix
dw.DimProduct	product_key, product_id, original and English category
dw.DimSeller	seller_key, seller_id, city, state, ZIP prefix
dw.DimDate	date_key YYYYMMDD, full_date, year, quarter, month, day
dw.FactPayments	Separate payment fact; PK and customer/date FKs verified in SSMS
dw.DimCategory	category_key, Portuguese category, English category; referenced by DimProduct

Sales means the sum of order-item prices; freight is measured separately. One order may contain multiple items, so order volume uses a distinct order count. Payments have a different grain and remain separate. The current product table retains its former category text columns while the mart moves to DimCategory; once all dependent SQL loads are updated, the redundant text can be removed after another validation run.

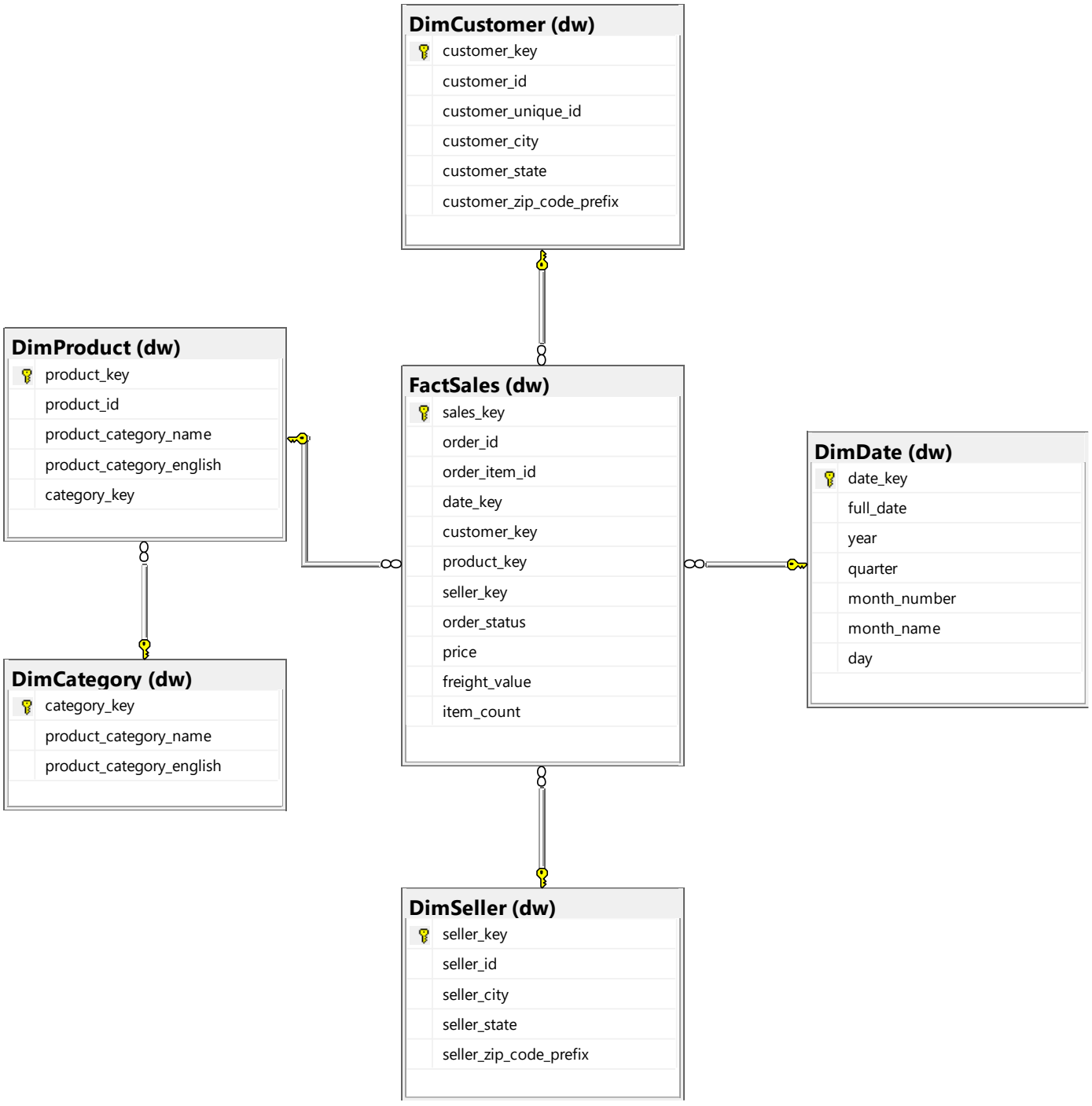


Figure 2 SnowFlake Schema

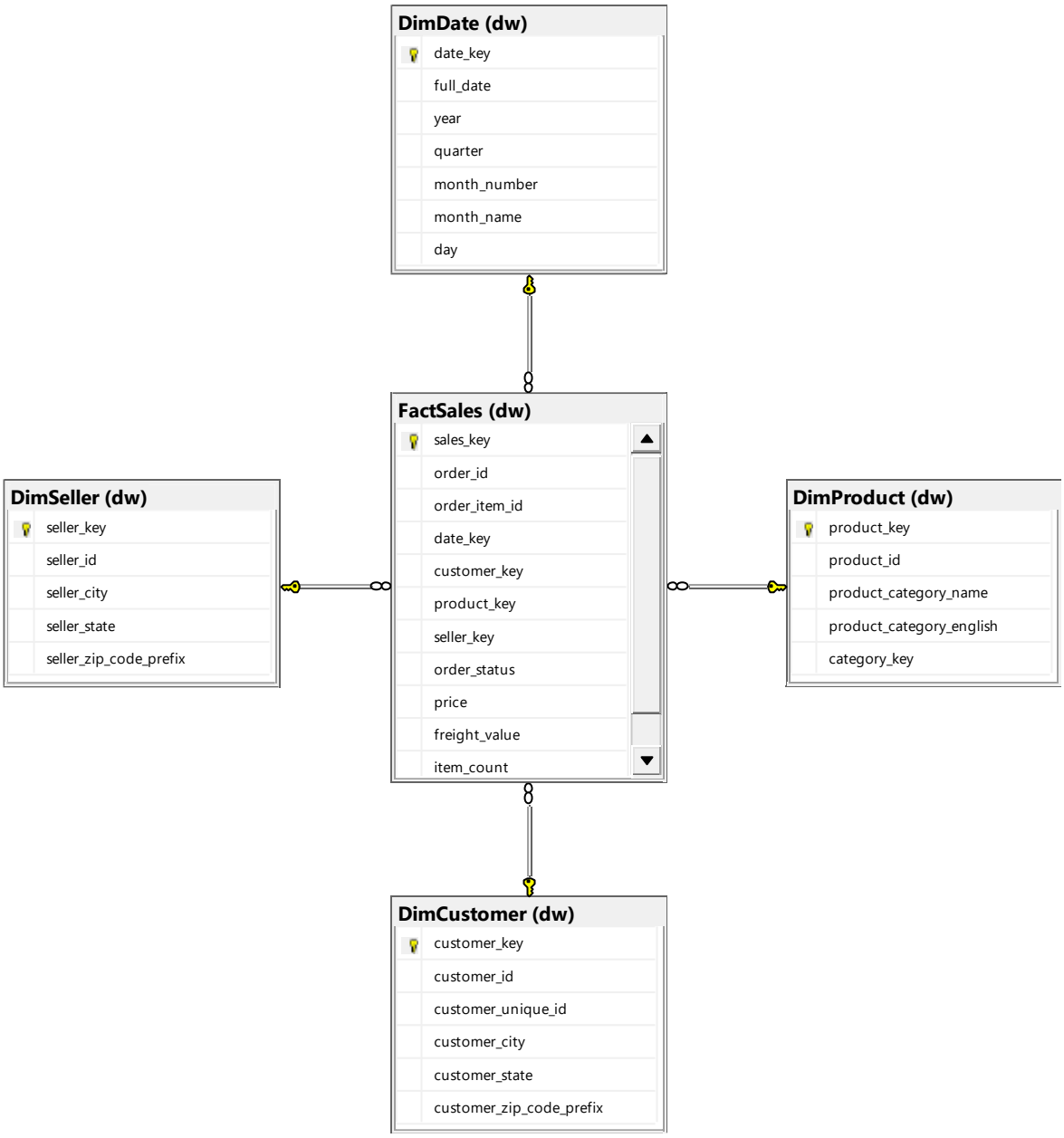


Figure 3 Star Schema

Design assumption	Explanation
Sales are stored at order-item level	Each row in dw.FactSales represents one item within an order, identified by order_id and order_item_id. An order containing several items therefore produces several fact rows.
Orders and items are different counts	The number of orders is calculated using distinct order IDs. Counting sales fact rows gives the number of order items.
Warehouse keys connect facts and dimensions	Customer, product, and seller dimensions use surrogate keys for warehouse relationships. Original source identifiers are retained for matching and traceability. The date key uses the YYYYMMDD format.
Purchase date is the main analysis date	Sales are analysed according to the order purchase date, rather than delivery or payment date.
Sales value excludes freight	Item price represents sales value. Freight is stored and analysed separately. Neither measure represents profit.
Delivered KPIs include delivered orders only	Delivered Sales, Delivered Orders, Delivered Items, and Delivered Freight filter records to order_status = 'delivered'. Other statuses remain available for status analysis.
Missing categories remain in the analysis	Missing product categories are labelled Unknown. When an English translation is unavailable, the original category name is retained.
Customer location is taken from the source record	Customer city and state represent the location recorded for the order-linked customer ID. The customer dimension does not provide a history of address changes.

5 ETL and checks

ETL workflow and loading order

1. Extract the selected CSV files with SSMS Import Flat File into dbo.stg_tables. Keep the original archive unchanged.

2. Inspect staging row counts, sample fields, nulls and key matches. Correct the category translation header issue and column types before dimension loads.

3. For a repeatable snowflake load, populate DimCustomer, DimSeller, DimDate and DimCategory before DimProduct. Assign category_key in DimProduct before loading FactSales. Verify the saved ETL script uses this updated order.

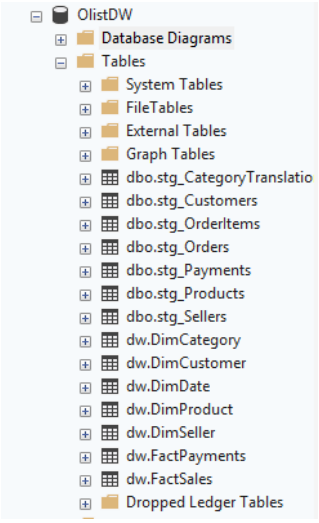
4. Load FactSales from order items after matching order_id to purchase date and customer and matching product and seller IDs to their dimensions. FactPayments, where loaded, follows its separate payment grain.

5. Compare source, staging, dimensions, FactSales and mart results; investigate missing matches, duplicate order items and total differences. Refresh Power BI after changing the view.

Transformation logic

Transformation	**Implementation**	**Purpose**
Missing value handling	NULLIF converted blank location values to NULL. COALESCE supplied category fallback values.	Handle incomplete attributes without unnecessarily excluding records
Data cleansing	LTRIM and RTRIM removed surrounding spaces from identifiers and text fields.	Improve consistency and matching
Standardisation	UPPER standardised customer and seller state codes. Category translations provided English names.	Support consistent regional and category analysis
Data type conversion	TRY_CONVERT converted ZIP prefixes to integers and purchase timestamps to dates.	Store attributes in suitable data types
Duplicate prevention	NOT EXISTS prevented insertion of dimension identifiers already present in the warehouse.	Avoid duplicate dimension records when loading scripts were rerun
Unique date creation	DISTINCT selected unique purchase dates.	Create one dimension record for each purchase day
Derived attributes	Date key, year, quarter, month number, month name, and day were calculated from purchase dates.	Support time-based filtering and hierarchies

SSMS Import Flat File loaded the six sales CSVs into dbo.stg_Orders, stg_OrderItems, stg_Customers, stg_Products, stg_Sellers and stg_CategoryTranslation; payments was staged separately. The ETL cleaned identifiers and locations, used TRY_CONVERT for dates and numbers, handled missing category descriptions and used the translation mapping. Customer, seller, date, category and product dimensions are loaded before FactSales, which resolves their keys. The category snowflake changes future product loads: DimCategory must be populated before DimProduct.category_key is assigned.



Validation was carried out at three levels. The supplied files contain 99,441 orders, 112,650 item rows and 634 distinct valid purchase dates. Independently calculated delivered-item totals are 110,197 items, 96,478 distinct orders, 13,221,498.11 item-price sales and 2,198,275.64 freight. The delivered-sales Power BI cards previously showed these values rounded to 13.22M, 96K and 110K, with freight displayed as 2,198,275.64. SSMS screenshots supplied for this project show PKs on DimCustomer, DimDate, DimProduct, DimSeller, FactSales and FactPayments, four FactSales FKs, and customer/date FKs on FactPayments. Final FactSales and mart row counts, payment totals and the new DimProduct → DimCategory .

Validation query or test	**Expected or group result**
SELECT COUNT(*) FROM dw.DimDate	634 reported in SSMS; 634 distinct source purchase dates.
SELECT COUNT(*) FROM dw.FactSales	Source benchmark: 112,650 items; final SSMS result to insert.
Delivered item count, order count, SUM(price), SUM(freight_value)	Source: 110,197 items; 96,478 orders; sales 13,221,498.11; freight 2,198,275.64. Compare to final SSMS output.
Fact PK or unique order/item and foreign key checks	PK and existing FactSales FKs verified in SSMS screenshots; duplicate-query result and category FK screenshot to insert.
Payment fact count and total, if implemented	FactPayments PK and customer/date FKs shown; row count and monetary totals to insert.

Loading sequence

- Import staging tables.
- Load customer, seller, date and category dimensions.
- Load products with their category keys.
- Load FactSales after resolving its dimension keys.
- Load FactPayments through its separate path.
- Create or update the sales mart.
- Refresh Power BI.

Validation results

- Source and staging row counts
- Dimension and fact row counts
- Duplicate business-key checks
- Missing required-key checks
- Unmatched dimension/foreign-key checks
- Invalid conversion checks
- Sales and freight total comparisons
- Mart versus fact comparisons

Check	Source-based result
Orders	99,441
Order-item rows	112,650
Distinct valid purchase dates	634
Delivered item rows	110,197
Distinct delivered Orders	96,478
Delivered item-price sales	13,221,498.11
Delivered freight	2,198,275.64

6 Sales analysis mart

mart.SalesAnalysis serves the sales and marketing analysis area. Its item rows combine FactSales with customer location, seller location, purchase date and product category. The category is reached through DimProduct → DimCategory in the snowflake version of the view. Analysts can compare sales by month, state or category while preserving the source item grain. This view is an analytical data mart; FactPayments remains separate from its measures.

Target users

Sales managers, marketing analysts and other staff reviewing sales performance.

Implementation

Explain that it is a SQL view joining FactSales to the relevant dimensions. Include its actual CREATE VIEW definition or a readable excerpt.

The fields it exposes, such as:

- Order and item identifiers
- Status
- Price and freight

- Purchase date and date attributes
- Customer state
- Seller state
- Product category

Analytical benefits

Explain that users can compare sales by category, time and location through one reporting view, while retaining the item grain.

7 Power BI report

Power BI Desktop imports **mart.SalesAnalysis from OlistDW**. We used DAX measures for Delivered Sales (sum of item price for delivered orders), Delivered Orders (distinct order_id for delivered orders), Delivered Items (count of delivered item rows), Delivered Freight (sum of freight for delivered rows) and Average Order Value (delivered sales divided by delivered orders). The delivered KPI cards previously displayed 13.22M in sales, about 96K orders, 110K items and 2,198,275.64 in freight; final screenshots should show the complete pages and the filters used.

Report page	Visuals and interaction to document
Executive Summary	Four delivered KPI cards; product-category and customer-state sales bars; distinct orders by status; year slicer if retained.
Trend Analysis	Monthly delivered sales and orders lines with Month Start axis; yearly sales comparison; label partial first and last years clearly.
Interactive Analysis	Year, state and category slicers; freight comparison; state-category matrix; year/quarter/month drill and roll-up; category bubble chart.

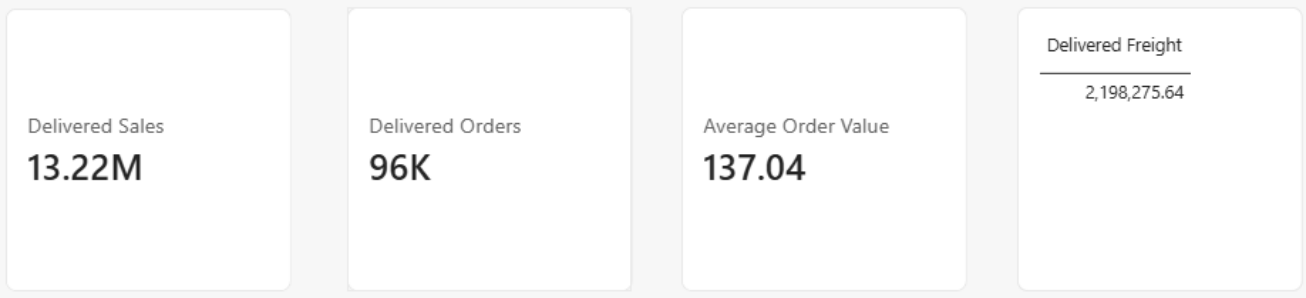


Figure 4 KPI Summary Page

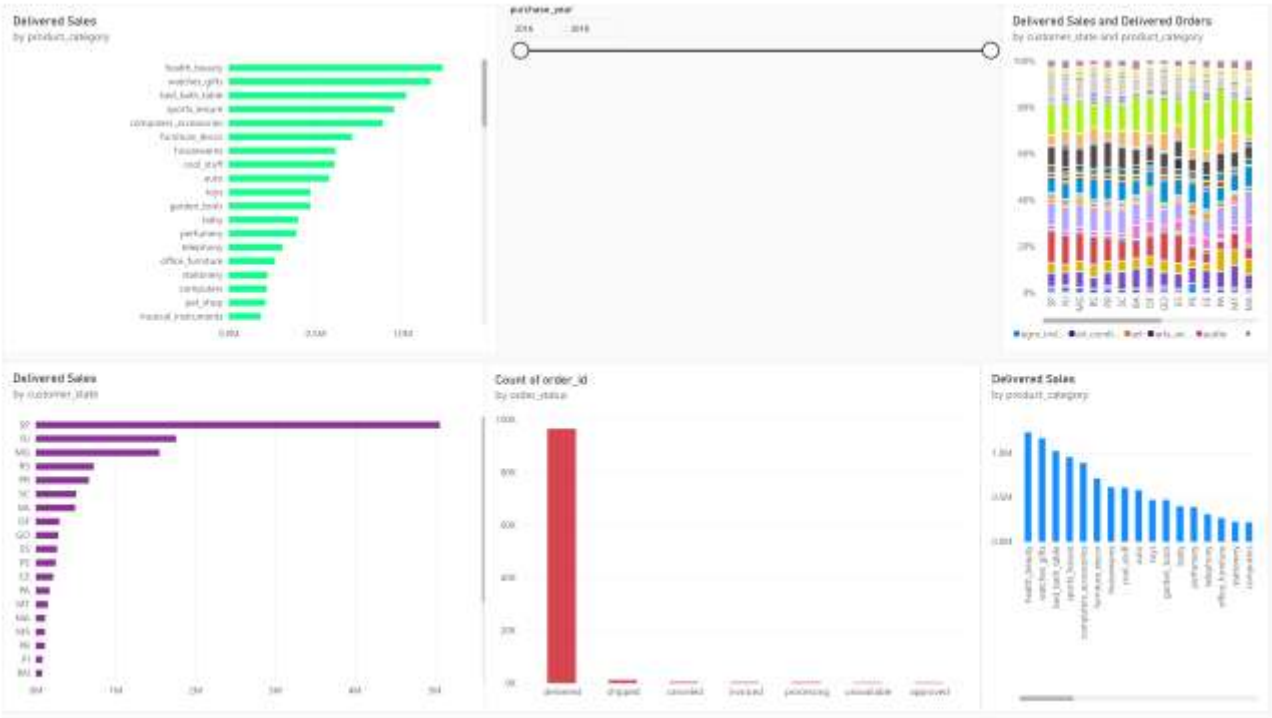


Figure 5 Executive Summary Page

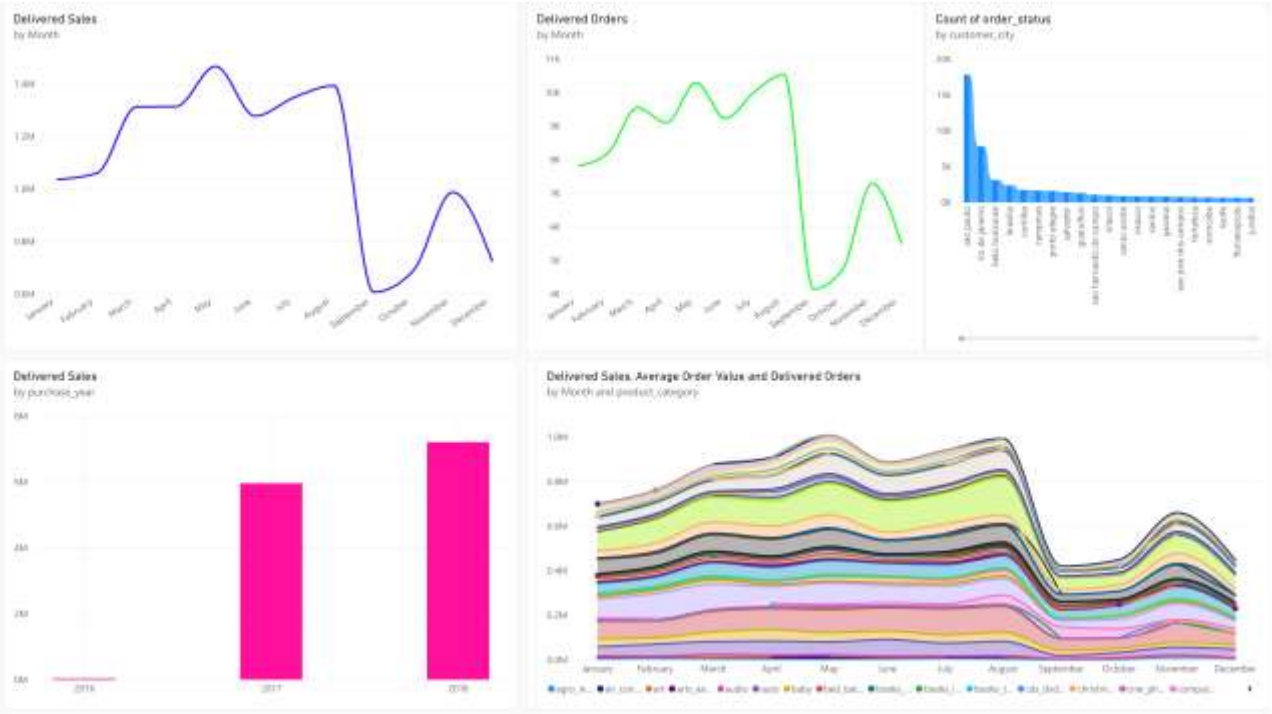


Figure 6 Trend Analysis Diagram

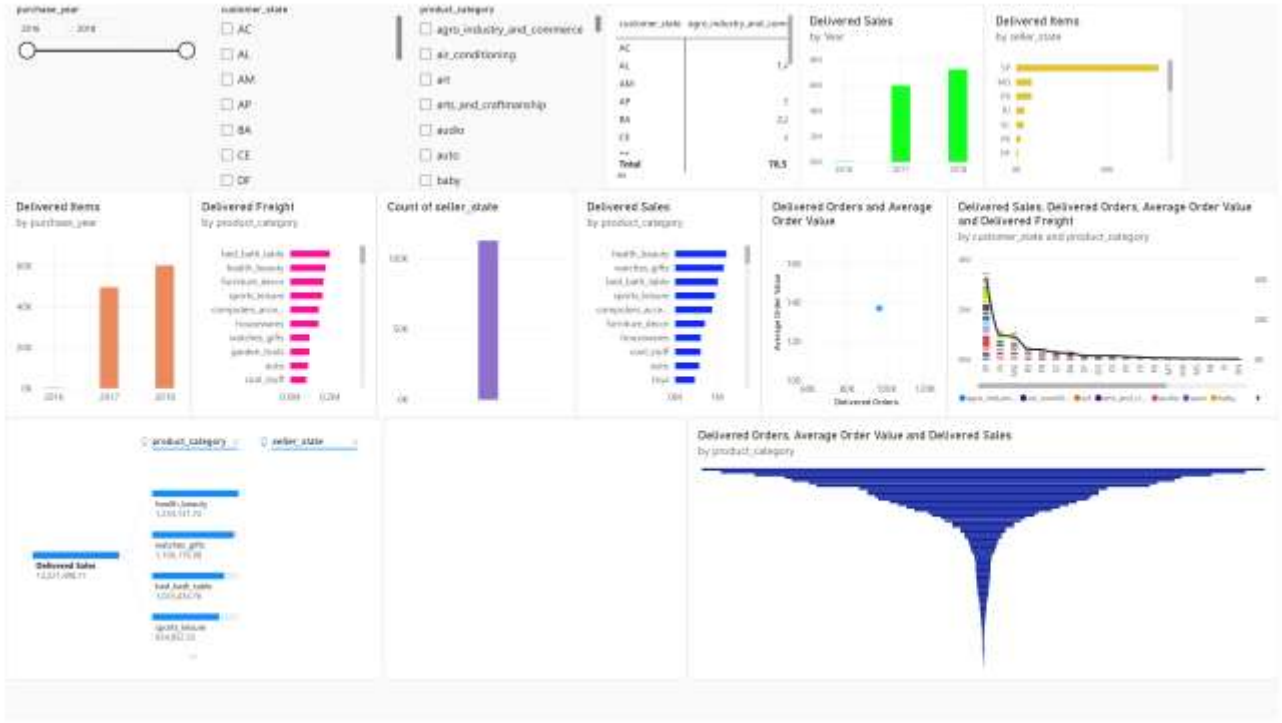


Figure 7 Interactive Analysis Diagram

8 Findings and recommendations

The five observations below were calculated from the supplied operational files using the same delivered-item definitions as the dashboard cards. They should be checked against the saved Power BI pages with slicers cleared. We compare the full months of 2017 for the time insight because the dataset begins partway through 2016 and ends during 2018.

Insight topic	Evidence to fill from your charts	Decision to justify
Sales over time	November 2017 delivered sales: 987,765.37; October: 648,247.65; December: 726,033.19.	Prepare capacity and stock ahead of the late-year demand peak; compare additional years before assuming seasonality.
Products	Health and beauty delivered sales: 1,233,131.72; watches and gifts: 1,166,176.98.	Give the leading categories sufficient stock and monitor whether promotion increases order volume.
Customer regions	São Paulo (SP) delivered sales: 5,067,633.16; Rio de Janeiro (RJ): 1,759,651.13.	Prioritize fulfillment capacity in SP and assess demand in other states before expanding campaigns.
Order outcomes	96,478 orders were delivered; 625 orders have canceled status in the source orders file.	Review cancellation reasons separately; source order statuses include orders without item rows.
Freight by category	Bed and bath table freight: 201,774.50; health and beauty: 178,957.81.	Review shipping costs and product packaging in the high-freight category.

Group contribution and submitted files

The group should replace the proposed allocation below with each member's actual work and student ID.

Member	Suggested responsibility	Evidence
IT24102286	Dataset profiling, source mapping, Task 1 and 2	Source inventory and preparation notes
IT24104273	Architecture and SQL star schema, Task 3 and 4	Diagrams and DDL
IT24102567	ETL and sales mart, Task 5 and 6	Staging and load scripts, validations
IT24103864	Power BI, insights and integration, Task 7 and 8	PBIX, dashboard screenshots, conclusions

Use of AI tools

Our group used ChatGPT for explanations of the assignment, help diagnosing SQL and Power BI errors, and editing the report structure and language. We carried out the imports, SQL work and dashboard construction ourselves. We will check the final technical details, figures and recommendations against our own files before submitting.

References

- Course assignment: IT3101 Design and Implementation of a Data Warehouse and Business Intelligence Solution, supplied PDF.
- https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce?utm_source=chatgpt.com